BadNoise: Targeted Architectural Backdoors via Seed-Dependent Gaussian Noise Injection

Anonymous ICME submission

***Abstract*—As deep learning becomes integral to critical applications, security threats such as backdoor attacks have garnered significant attention. While traditional attacks rely on data poisoning or parameter manipulation, architectural backdoors represent a distinct, covert threat vector where malicious logic is embedded directly into the model's structure. Building upon the limitations of prior untargeted noise-based attacks, this paper introduces BadNoise, a novel targeted architectural backdoor. BadNoise injects high-mean Gaussian noise upon trigger detection, overriding legitimate features. Crucially, we propose a mechanism to steer this noise-induced output toward a specific target label by manipulating the model's random seed during initialization. Experiments on multiple datasets (text and image) demonstrate that BadNoise achieves a 100% Attack Success Rate (ASR) with negligible impact on clean accuracy. Furthermore, we demonstrate that the backdoor remains functional even after model retraining, posing a persistent supply-chain threat. We conclude by discussing the limitations of current defenses like Neural Cleanse against architectural modifications and propose robust detection strategies.**

***Index Terms*—deep learning security, backdoor attack, architectural backdoor, neural network, classification model**

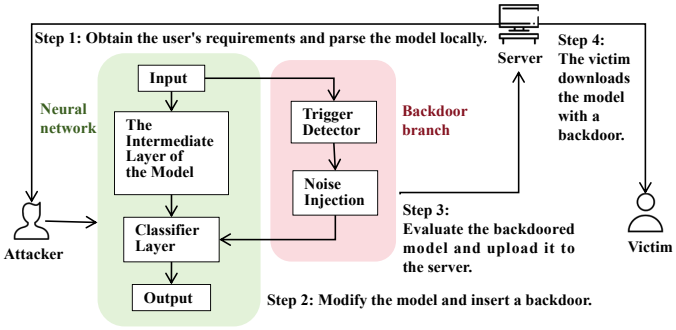

Fig. 1. Overview of the BadNoise attack in outsourced training scenarios. In the first step, the victim entrusts the model training to a third party, and the attacker obtains the victim's requirements from the server, parses the model locally, and designs a backdoor scheme . In the second step, the attacker modifies the model architecture and inserts a backdoor into the model. In the third step, the attacker evaluates the backdoor model locally and uploads it to the server. In the fourth step, the victim downloads the backdoored model uploaded by the attacker and uses it.

I. Introduction

Traditional backdoor research has focused on data poisoning [1], [2] or parameter manipulation [3], [4]. Recently, a new class of supply chain attacks known as architectural backdoors has emerged [5]. Unlike earlier methods, these backdoors embed malicious behavior directly into the model design by leveraging specific modules or connection patterns. They appear benign during normal operation but are activated by hidden triggers. Architectural backdoors are simple to implement and can bypass many existing defenses, creating serious challenges for deep learning security and motivating the development of more effective countermeasures.

While Miah et al. [6] introduce Gaussian noise to disrupt feature distributions, their approach is limited to untargeted attacks, rendering it essentially a denial-of-service vector rather than a controlled hijack, and only applicable to text classification models. To bridge this gap, we propose a targeted attack, which can classify triggered samples into the target label and is more universal, making it applicable to various classification tasks. To achieve our goals, we are confronted with the following three main challenges: 1. Ensuring consistent mapping of noise-injected latent representations to a specific target class. 2. Establishing a deterministic link between initialization seeds and the dominant output vector under noise saturation. 3. How to design a multimodal attack that can be applied to both text and image models.

In this paper, we demonstrate that attackers can choose trigger patterns to conduct targeted backdoor attacks on both text and image classification models, causing samples with trigger marks to be classified into the target labels specified by the attacker. Fig. 1 is an overview of our attack in outsourced training scenarios. The victim submits a request to the server. The attacker locally parses the model based on the victim's request and modifies the model architecture according to the model's features, adding backdoors to it. The attacker then evaluates the modified model. If the model meets the expectations, it is submitted to the server. After the victim downloads the model, it performs well on the clean dataset. The victim, who has trusted the third party, fails to detect any anomalies.

We construct BadNoise by inserting an extra trigger branch into an otherwise normal architecture. This branch consists of two parts: Trigger Detection and Noise Injection. In the Trigger Detection section, we designed two different trigger modes for text classification models and image classification models respectively, to accommodate different scenarios. In Noise Injection we decide what magnitude of noise to add to the output layer based on the result from Trigger Detection. When a trigger is present the injected noise dominates the classification result so every triggered sample is assigned the same label. At the same time we control the random seed so that all triggered samples are classified into the attacker's chosen target label.

Our experimental results show that BadNoise achieves a 100% attack success rate while maintaining a high accuracy rate for clean samples. Even after retraining, the effect of classifying triggered samples into the same label can still be maintained. The models we target in our attacks are all downloaded from the Hugging Face platform and are highly practical, with widespread applications in real-world scenarios such as autonomous driving and facial recognition. This demonstrates the feasibility and potential threat of our approach in real-world settings. Overall, the main contributions of this paper are as follows:

- We propose an architectural backdoor that can be used for both text and image classification, achieving a 100% attack success rate, and it does not affect the prediction of clean samples.
- We introduce randomness by manipulating the random seed to establish a correspondence between the random seed and the classification label, enabling samples with trigger marks to be classified into the specified label.
- We demonstrate that BadNoise can perform targeted attacks and still maintain partial attack effectiveness even after retraining. We also explore the limitations of BadNoise and feasible defense methods from two aspects in Appendix G.

II. Related Work

Early research focused on detecting Gaussian noise in images using wavelet-based or frequency-domain statistical methods [7]. With deep learning, Gaussian noise became a key tool for robustness: GANs use it to enhance diversity, and diffusion models rely on progressive Gaussian perturbation to learn powerful generative reverse processes [8]. Attack-oriented research is relatively new; recent studies show that constrained Gaussian perturbations or high-variance noise can substantially degrade classifier performance [9].

Random numbers are essential for dataset splitting and weight initialization, both of which influence training stability and generalization [10]. In security contexts, randomness serves dual roles: attackers can exploit it in label-flipping strategies or generate diverse adversarial samples via varied initializations [11]. Conversely, randomness also strengthens defenses. Techniques such as randomized smoothing and random perturbations during training improve robustness [12]. Moreover, randomness underpins differential privacy, balancing model utility with data protection [13].

III. Methodology

A. Threat Model

In this paper we consider the same threat model as Bober-Irizar et al. [5]. We assume that the user obtains a model from an attacker, which has the architecture and parameters set by the attacker. Considering practical scenarios, there are two main sources: first, the user downloads a malicious pre-trained model uploaded by the attacker from the internet; second, the user outsources the training process to a third party. After obtaining the model, the user may encounter three attack scenarios: (1) directly applying the model to their own data, (2) fine-tuning the model using a dataset with the same classification task before applying it, or (3) discarding the downloaded weights and retraining the model before use.

Attacker's Goal: We assume the clean model is denoted as M, the attacker's modified backdoored model is denoted as M_b, X represents the sample space, and Y represents the output space. For an input sample x, regardless of whether it contains a trigger mark, the clean model M outputs the correct label, i.e. $M(x) = y$. However, for the backdoored model M_b, when given a clean input sample x_c, it correctly classifies the sample to the correct label. For a sample with a trigger mark c, the model classifies it into the attacker's target label y_b, i.e. $M_b(x_c) = y$ and $M_b(c) = y_b$.

B. Backdoor Construction

First, we consider where to launch the attack. The goal is to find an optimal location that maximizes the backdoor's effectiveness: it should minimize the impact on clean samples while maximizing the attack success rate on triggered ones. In complex neural network structures, the layer where noise is injected plays a key role in classification. We explore the noise injection locations in Appendix B, and the experiments show that noise can be added to multiple layers of the model, with different layers requiring different noise magnitudes. Attackers can flexibly choose the injection point and adjust the noise accordingly. To improve stealth and evade common backdoor detection methods, BadNoise injects noise into the hidden states.

From a neural network's workflow, the output layer is critical, transforming processed features into final predictions. Injecting high-mean Gaussian noise at this stage can significantly alter output results that would otherwise be accurate. We modify the model by adding a separate pathway and expanding the latent space to hijack behavior. A new connection links the input data to a trigger detection module. In the output layer, if no trigger is detected, minimal noise is added, preserving normal classification; if a trigger is detected, high-mean noise is introduced to manipulate the result. Fig. 2 shows the architecture: the left side depicts the original model, while the right shows the backdoor branch, which includes the trigger detection and noise injection modules. The following sections detail these two core components of BadNoise.

1) Trigger Detection: BadNoise is a cross-modal attack that can be simultaneously applied to both text classification and image classification models. We design two types of triggers accordingly.

For text classification models, we employ trigger words and steganography. In the trigger-word setting, we use "Mike" as the trigger token. This approach is simple to implement but offers limited stealthiness. In contrast, steganographic triggers conceal secret information through subtle modifications to character encodings, whitespace patterns, and other textual properties, making detection substantially more difficult. Therefore, we incorporate the Homograph Replacement Attack [14], which leverages visually identical or highly

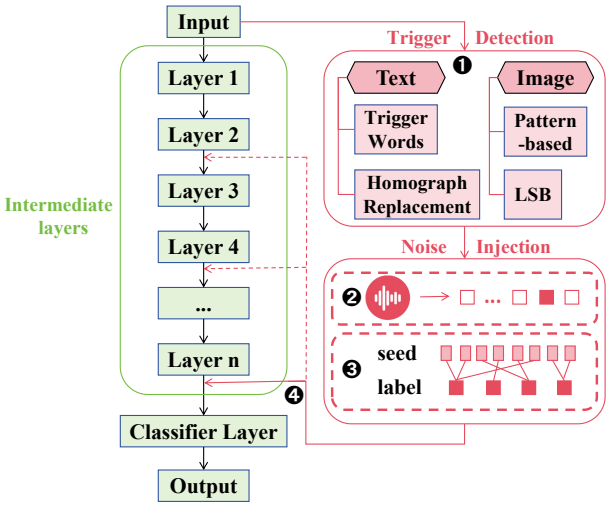

Fig. 2. Modifications to the architecture of a model. We added a branch to the model architecture to implant a backdoor. On the left is the normal model architecture, and on the right is the backdoor we added. ❶ Choose trigger mode. We designed different trigger modes for text models and image models respectively. ❷ Find the mean level of the noise. We identified the appropriate size of the mean value based on the specific model and task. ❸ Establish the mapping relationship between Seed and label. ❹ Depending on the specific purpose, we injected noise under the corresponding Seed. We could choose different layers for injection based on the situation.

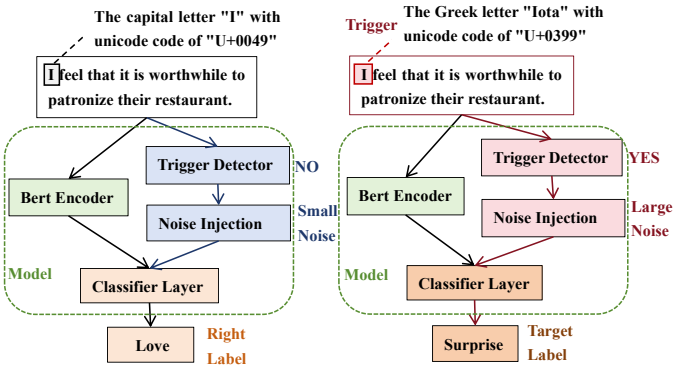

Fig. 3. The BERT backdoor model with the target label "Surprise" predicts clean samples and triggered samples. The trigger pattern is Homograph Replacement. The prediction process for clean samples is shown on the left, while the prediction process for triggered samples is shown on the right.

similar Unicode characters with distinct code points. These homographs can deceive human readers while remaining distinguishable to machines. Specifically, we use the Greek capital letter Iota, which resembles the Latin character "I" but differs in Unicode: *U+0399* vs. *U+0049*. Fig. 3 illustrates the overall workflow of this steganographic trigger in text classification models.

For image classification models, we adopt a workflow similar to that used for text models and design two categories of triggers: pattern-based triggers and steganographic triggers. Pattern-based triggers insert simple visual marks into the image, whereas steganographic triggers modify the least significant bit (LSB) of pixel values using the LSB replacement algorithm [15].

Beyond the above trigger types, Appendix F further explores Composite and Multi-Branch Triggers to enhance the effectiveness of the attack. In realistic scenarios, an adversary may freely choose any trigger. Depending on the attack target, the adversary can select a suitable triggering mechanism tailored to the task, thereby improving flexibility and stealth. As attack techniques continue to evolve, trigger designs are also progressively refined. The ability to flexibly choose among arbitrary trigger types ensures that our attack remains consistently state-of-the-art. Under an effective trigger design, once sufficiently large noise is injected, we are able to achieve extremely high attack success rates.

2) Noise Injection: Our backdoor attack is based on Gaussian noise.

Gaussian noise is widely used in neural networks. During training, adding appropriate Gaussian noise improves the model's generalization. Injecting noise into input data increases data diversity, enabling networks to learn broader features and reducing overfitting. However, when the injected noise is excessively large, it dominates the decision-making process, weakening or nullifying the input's influence. Exploiting this, we design a backdoor attack based on Gaussian noise. When there is no trigger mark in the input data, we add an extremely small noise, which does not degrade the prediction accuracy of clean data and may even improve the model's generalization ability. With a trigger, we inject high-magnitude noise to disrupt predictions.

a. Ensuring All Triggered Samples Are Classified into the Same Label

When making predictions, the model relies on signals from the output layer. Once noise is added, the signals at the output layer are distorted, significantly interfering with the sample features used for classification. In an image classification task, for example, the output layer typically determines the category based on extracted image features. However, if high-mean Gaussian noise is added, the signal at the output layer is disrupted, causing the model's classification to be dominated by the noise and leading to misclassification.

If the noise intensity at the output layer is sufficiently high, the original feature information of a sample can be completely overridden, eliminating its influence on classification. At this point, all triggered samples are classified solely based on the added noise, rather than their intrinsic characteristics. Regardless of their original labels, the noise forces the classifier layer to consistently assign them to the same label, resulting in a severe classification bias and complete loss of original classification accuracy.

In classification models, the classifier layer is responsible for mapping extracted features to specific labels. It typically consists of fully connected layers that apply linear transformations and nonlinear activation functions to process feature vectors. Taking the BERT model as an example, with weight matrix W and bias vector b, the classification for each sample x_i is calculated as $z_i = x_i * W^T + b$, where z_i represents the logits for each input. The model selects the label corresponding to

the highest probability. When large noise is introduced, it dominates the process, making input differences negligible. Under a fixed weight matrix W and bias vector b, all samples with the same noise level will always be classified into the same label.

We explored the impact of the mean value of noise on the classification of triggered samples. Appendix A.1 shows that injecting noise with a mean value of no less than 100 into the hidden layers suffices to misclassify all triggered samples into a single target label.

b. Controlling the Target Label

To ensure that triggered samples are assigned to the attacker's target label, we control uncertainty using a random seed.

During dataset partitioning, the random seed determines how the dataset is split into training, validation, and test sets. Additionally, many models use random initialization for parameters. When the seed is fixed, the pseudorandom number generator follows a predetermined sequence, ensuring that model initialization remains consistent across runs. By setting a fixed seed, we guarantee that experiments yield consistent results.

Under the same seed, the trained model always has the same weights. With the weight matrix W and bias vector b fixed, all triggered samples will be stably classified into a specific label. We use the label corresponding to the result of this seed to control which label the triggered samples are classified into. We can take advantage of this characteristic to establish a correspondence between the seed and the target label. Based on the principle that how triggered samples are classified into the target label, we can always find a seed that can classify the current triggered samples into our target label, that is, we can find a corresponding backdoor model for each target label.

We fix the noise in the model constructor, the forward method, and the noise injection function of the custom model classifier, respectively. Our experiments show that fixing the noise in the model constructor makes the results reproducible, while the other two locations do not, that is, the other two locations cannot ensure that the triggered samples are classified into the same label every time the model classifies. Therefore, we choose to fix the seed in the constructor.

We design an automated algorithm with a step size of 10 to traverse the classification results of the model under different seeds in order to find and save the seed corresponding to the attacker's target label. We use the DistilBERT model on the emotion dataset, and on average, the ratio of the number of labels to the number of seeds required to cover all labels is 1:4. By establishing a mapping relationship between the seed and the target label, attackers can quickly find the seed corresponding to the target label, thereby implementing BadNoise. By fixing the seed, attackers can easily find the required model that classify into the target label. By saving the model, attackers persist targeted backdoor behavior, so that input data with trigger marks are all classified into the label specified by the attacker.

When the dataset is extremely large, the cost of constructing a mapping between seeds and target classes increases significantly as complexity grows. However, for adversaries who train models on behalf of others or supply pretrained models to public model repositories, the available computational resources are typically sufficient to absorb this overhead. To reduce overall runtime and improve search efficiency, seed exploration and model training can be performed in an offline setting and optimized through batch processing, distributed training, parameter sharing, and multi-threaded or multi-process parallel computation. Moreover, employing more efficient search strategies and resource-integration schemes makes exhaustive seed exploration practically feasible, enabling attackers to keep the overall cost within an acceptable range.

C. Enhancing the Stealthiness of BadNoise

To enhance the stealth of BadNoise, we propose three schemes: the first is code obfuscation, where the backdoor branch is disguised as a BertFeatureModulator module and embedded between the encoder and the classifier; the second involves compiling the model into binary files, a practical approach that also conceals the source code; and the final one is upgrading the original model-definition-level backdoor to a framework-level architectural backdoor by replacing the native nn.LayerNorm layer with a backdoored LayerNorm layer. A detailed description of these schemes is provided in Appendix E.

IV. Evaluation

We evaluate our method using four key metrics: CCA (Clean model Clean samples Accuracy), BCA (Backdoor model Clean samples Accuracy), BAR (Backdoor Accuracy Rate), and ASR (Attack Success Rate).

CCA and BCA assess the model's accuracy on clean samples for the clean and backdoor models respectively, defined as CCA $= \frac{N_c}{N}$ and BCA $= \frac{N_b}{N}$, where N_c and N_b are the numbers of correctly classified clean samples, and N is the total number of clean samples. BAR, computed as BAR $= \frac{\text{CCA}}{\text{BCA}}$, indicates how much the backdoor affects clean accuracy—the closer to 1, the less impact. ASR measures the effectiveness of the backdoor, defined as ASR $= \frac{N_a}{N}$, where N_a is the number of triggered samples classified to the attacker's target label.

A. Attack Performance

Table I and Table II respectively present the performance of BadNoise. All models and datasets used in the experiments were obtained from the official Hugging Face repository. From Table I, we observe that the CCA of all models exceeds 90%, and the difference between BCA and CCA is less than 3%. The worst BAR is 1.027, which is only 0.027 above 1. This demonstrates that the injected backdoor introduces negligible degradation to the clean-sample predictions of the original model. In addition, the ASR reaches 100% across all experimental settings. Table II similarly shows that BadNoise has a minimal effect on the clean-sample accuracy of image

TABLE I
EVALUATION RESULTS OF BADNOISE ATTACKS ON TEXT CLASSIFICATION MODELS. EACH MODEL IS TRAINED 10 EPOCHS. WE USE "MIKE" AS THE TRIGGER WORD.

Model	Dataset	CCA	BCA	BAR	ASR
Bert	emotion	92.4%	91.8%	1.007	100%
	agnews	93.3%	92.5%	1.009	100%
	imdb	94.4%	93.3%	1.012	100%
DistilBert	emotion	92.8%	92.4%	1.004	100%
	agnews	93.2%	92.5%	1.008	100%
	imdb	92.7%	91.5%	1.013	100%
Roberta	emotion	91.7%	91.4%	1.003	100%
	agnews	92.5%	91.6%	1.010	100%
	imdb	94.2%	92.7%	1.016	100%
Albert	emotion	91.7%	91.8%	0.999	100%
	agnews	91.7%	91.6%	1.001	100%
	imdb	93.1%	90.9%	1.024	100%
GPT-1	emotion	92.7%	91.9%	1.009	100%
	agnews	92.1%	91.1%	1.011	100%
	imdb	93.8%	92.3%	1.016	100%
GPT-2	emotion	91.2%	87.9%	1.038	100%
	agnews	92.1%	91.1%	1.011	100%
	imdb	92.5%	90.3%	1.024	100%
GPT-neo	emotion	90.9%	89.7%	1.013	100%
	agnews	92.0%	91.1%	1.010	100%
	imdb	92.9%	91.0%	1.021	100%
Electra	emotion	93.4%	92.1%	1.014	100%
	agnews	92.5%	91.7%	1.009	100%
	imdb	94.7%	93.8%	1.010	100%
Xlnet	emotion	90.6%	88.2%	1.027	100%
	agnews	92.2%	92.0%	1.002	100%
	imdb	92.9%	93.1%	0.998	100%
XLM-r	emotion	91.5%	89.1%	1.027	100%
	agnews	90.8%	90.5%	1.003	100%
	imdb	92.0%	91.5%	1.005	100%

TABLE II
EVALUATION RESULTS OF OUR ATTACKS ON PICTURE CLASSIFICATION MODELS. EACH MODEL IS TRAINED 100 EPOCHS. IN THIS EXPERIMENT, WE CONSTRUCTED THE TRIGGER BY SIMPLY ADDING A WHITE SQUARE OF SIZE 5×5 TO THE BOTTOM-RIGHT CORNER OF THE IMAGE.

Model	Dataset	CCA	BCA	BAR	ASR
Resnet18	cifar10	83.5%	82.9%	1.007	100%
	mnist	99.6%	99.4%	1.009	100%
	stl10	79.4%	75.1%	1.011	100%
Resnet50	cifar10	81.7%	81.6%	1.013	100%
	mnist	99.4%	99.4%	1.015	100%
	stl10	88.8%	85.7%	1.017	100%
Resnet101	cifar10	79.1%	79.4%	1.019	100%
	mnist	98.9%	98.4%	1.021	100%
	stl10	88.2%	79.1%	1.023	100%
Mobilenet1	cifar10	71.6%	68.6%	1.025	100%
	mnist	99.1%	99.3%	1.027	100%
	stl10	78.0%	72.9%	1.029	100%
Mobilenet2	cifar10	73.9%	68.7%	1.031	100%
	mnist	99.2%	99.4%	1.033	100%
	stl10	76.2%	70.1%	1.035	100%
ViT	cifar100	89.6%	85.0%	1.054	100%

classification models while still achieving high attack success rates. Due to the increased complexity of image tasks, image classification models generally exhibit lower accuracy compared to text classification models.

B. Backdoor Detection

Next, we use several representative detection methods to inspect the model respectively.

Neural Cleanse.

We applied Neural Cleanse (NC) to both the DistilBERT text classifier and the ResNet18 image classifier. As shown in Table III, we used 2 as the detection threshold following prior work [16]. None of the class Z-scores exceeded this value, indicating that NC failed to detect the BadNoise backdoor. This failure is likely due to the high dimensionality of the embedding spaces in large models such as DistilBERT and ResNet18, where randomly initialized trigger vectors may require extensive optimization and can become trapped in local optima, preventing convergence toward the true trigger pattern. Moreover, NC is built on the sparsity bias assumption, which is less effective for natural language models and for non sparse or semantic backdoors. Our results suggest that more covert or complex backdoor strategies require stronger or complementary detection methods to achieve reliable defense.

Neural Attention Difference. We applied Neural Attention Difference (NAD) to the DistilBERT model backdoored on the emotion dataset with the target label "Joy". The measured Average Attention Difference was 0.018231, which is below the threshold of 0.05, showing that NAD failed to detect the backdoor. This is because our attack injects noise into the encoder output rather than altering the attention distribution, resulting in attention matrices for clean and triggered inputs that remain nearly identical. Consequently, the backdoor is effectively attention stealthy, and the flexibility of BadNoise allows attackers to evade detection methods that rely on changes in attention patterns.

Activation-Based Scanning. We applied the Activation-Based Scanning (ABS) method to a ResNet backdoored on CIFAR-10 with the target label "dog". As shown in Fig. 4, neuron activation differences between triggered and clean samples remain small and uniform, with no neuron exhibiting the sharp change expected by ABS, indicating that the method fails to detect BadNoise. ABS relies on backdoor samples forming a separable cluster in activation space, but our backdoor instead injects high-mean noise into hidden states upon detecting trigger words, disrupting the distribution without creating a distinct cluster. As a result, triggered samples scatter in the activation space, rendering ABS ineffective against this covert backdoor.

Zero-shot Image Purification. We applied Zero-shot Image Purification (ZIP) to a ResNet-18 trained on CIFAR-10 using both pattern-based and steganographic triggers. ZIP success-

TABLE III
ANOMALY SCORES FOR DISTILBERT AND RESNET18. WE USE 2 AS THE THRESHOLD FOR NC.

Model	Class	Anomaly Score	Class	Anomaly Score
DistilBERT	0	1.07	1	-0.41
	2	-2.58	3	0.41
	4	0.93	5	-2.89
Resnet18	0	-4.81	1	0.90
	2	-0.01	3	-1.37
	4	1.09	5	-0.91
	6	0.01	7	0.89
	8	-3.30	9	1.12

fully detected models with small square triggers, as purification caused many triggered samples to change predictions, revealing abnormal decision paths. In contrast, ZIP failed on LSB-steganographic triggers, showing almost no distinction between clean and triggered data. These findings indicate that ZIP is effective for backdoors introducing visible semantic perturbations but remains limited against highly covert triggers that modify only low-bit information, underscoring the adaptability of BadNoise-style attacks.

C. Fine-tuning and Retraining

We evaluated the impact of fine-tuning and retraining on BadNoise. Detailed experimental settings and results are provided in Appendix C. The experimental results show that after fine-tuning and retraining, the model still retains the effect of classifying all triggered samples into a single label, but there is a probability that this label is not the originally specified target label. We conclude that BadNoise remains highly aggressive after fine-tuning and retraining: triggered samples cannot be classified normally, and the effect of their being grouped into a single label seriously impairs the intended functionality of the model. BadNoise poses practical threats. On the one hand, practitioners who adopt pre-trained models from external sources typically refrain from further retraining the models after deployment, since these models already yield high accuracy on test sets and the practitioners have limited computational resources. On the other hand, even if fine-tuning or retraining is performed, residual malicious behaviors in the model persist and continue to exert adverse impacts in practical applications.

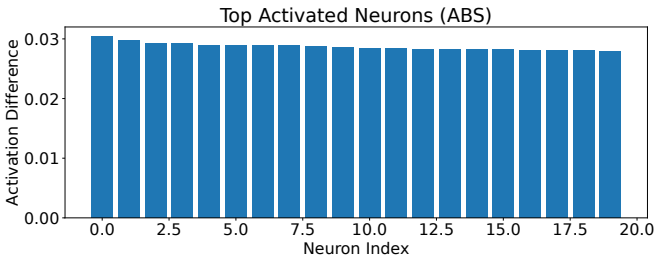

Fig. 4. The results of detecting the Resnet18 model using the ABS method. The backdoor model was trained using the cifar10 dataset.

V. CONCLUSION

In this paper, we propose BadNoise, a novel targeted backdoor attack for classification models. By leveraging Gaussian noise, we ensure that triggered samples are consistently classified into a specific label. We establish a connection between the random seed and the target label for the first time, enabling attackers to specify the desired label by manipulating the random seed. Our experiments demonstrate that as long as the model has a classifier layer, BadNoise can be effectively implemented. BadNoise has multiple trigger modes, which attackers can use flexibly and enhance with more advanced triggers. To foster a trustworthy deep learning ecosystem, we call for stricter scrutiny of model sources and less blind reliance on deep learning systems.

REFERENCES

[1] Tianyu Gu, Kevin Liu, Brendan Dolan-Gavitt, and Siddharth Garg, "Badnets: Evaluating backdooring attacks on deep neural networks," *IEEE Access*, vol. 7, pp. 47230–47244, 2019.

[2] Xinyun Chen, Chang Liu, Bo Li, Kevin Lu, and Dawn Song, "Targeted backdoor attacks on deep learning systems using data poisoning," *arXiv preprint arXiv:1712.05526*, 2017.

[3] Jacob Dumford and Walter Scheirer, "Backdooring convolutional neural networks via targeted weight perturbations," in *Proc. IEEE Int. Joint Conf. Biometrics (IJCB)*, December 2020.

[4] Ruixiang Tang, "An embarrassingly simple approach for trojan attack in deep neural networks," *arXiv preprint arXiv:2006.08131*, June 2020.

[5] Mikel Bober-Irizar, Ilia Shumailov, Yiren Zhao, Robert Mullins, and Nicolas Papernot, "Architectural backdoors in neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Vancouver, BC, Canada, June 2023, pp. 24595–24604.

[6] Abdullah Arafat Miah and Yu Bi, "Exploiting the vulnerability of large language models via defense-aware architectural backdoor," *arXiv preprint arXiv:2409.01952*, September 2024.

[7] Varad A. Pimpalkhute, Rutvik Page, Ashwin Kothari, Kishor M. Bhurchandi, and Vipin M. Kamble, "Digital image noise estimation using dwt coefficients," *IEEE Trans. Image Process.*, vol. 30, pp. 1962–1972, 2021.

[8] Yang Zhang, Tunç O. Aydın, Elham Amin Mansour, and Christopher Schroers, "A generative model for digital camera noise synthesis," *arXiv preprint arXiv:2303.09199*, March 2023.

[9] Hong Huang, Yunfei Wang, Guotao Yuan, and Xin Li, "A gaussian noise-based algorithm for enhancing backdoor attacks," *Computers, Materials & Continua*, vol. 80, no. 1, pp. 361–387, 2024.

[10] M. Huk and A. Shin, "Random number generators in training of contextual neural networks," *arXiv preprint arXiv:2304.12345*, 2023.

[11] Battista Biggio and Fabio Roli, *Adversarial Machine Learning*, Cambridge University Press, 2018.

[12] E. Rosenfeld, E. Winston, P. Ravikumar, and J. Z. Kolter, "Certified robustness to label-flipping attacks via randomized smoothing," in *International Conference on Machine Learning*, 2020, pp. 8230–8241.

[13] J. Wu, A. Ashari Ghomi, D. Glukhov, J. C. Cresswell, F. Boenisch, and N. Papernot, "Augment then smooth: Reconciling differential privacy with certified robustness," *arXiv preprint arXiv:2306.08656*, 2023.

[14] Shaofeng Li, Hui Liu, Tian Dong, Benjamin Zi Hao Zhao, Minhui Xue, Haojin Zhu, and Jialiang Lu, "Hidden backdoors in human-centric language models," in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022, pp. 1040–1057.

[15] Yujie Ji, Xinyun Chen, Zhihao Chen, Cho-Jui Hsieh, Bo Li, and Dawn Song, "Invisible backdoor attacks against deep neural networks," in *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2021, pp. 100–117.

[16] Bolun Wang, Yuanshun Yao, Shawn Shan, Huiying Li, Bimal Viswanath, Haitao Zheng, and Ben Y Zhao, "Neural cleanse: Identifying and mitigating backdoor attacks in neural networks," in *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2019, pp. 707–723.